

CSC 220 — LinkedList Implementation Activity

In-Class Practice + At-Home Completion

Individual work • Due Tuesday

Goal

In this activity, you will work with a partially implemented generic singly linked list. The provided code already defines `Node<T>` and `MyLinkedList<T>`, including construction of an empty list, insertion at the head, an empty-list check, and list traversal/printing.

Your goal is to understand how the nodes are connected, practice traversing the list, and implement additional operations by carefully changing node references and/or node data.

Provided Starter Code

You will receive `Node.java`, `MyLinkedList.java`, and `Demo.java`. Do not replace the implementation with `java.util.LinkedList`.

The current list is a singly linked list:

```
head → [data | next] → [data | next] → [data | null]
```

The provided `insert(Node<T> newNode)` method inserts at the head. Notice that each new node becomes the new head and points to the previous head.

Part 1 — Test and Understand the Starter Code

After the instructor demo, compile and run the provided `Demo.java`. Trace what happens to head and next each time a node is inserted.

- Run the provided Integer example.
- Add enough test values to make sure you can predict the order printed by `printList()`.
- Before moving on, make sure you understand the traversal pattern used inside `printList()`.

Key traversal pattern:

```
Node<T> temp = head;

while (temp != null) {
    // work with temp
    temp = temp.getNext();
}
```

Part 2 — In-Class Requirement: Complete ONE Group A Problem

During class, each of you will be assigned ONE problem from Group A. Complete and test your assigned method individually before the end of class.

Group A	Suggested Signature	Task
A1	<code>insertAtEnd(Node<T> newNode)</code>	Insert a new node at the end of the

		list.
A2	deleteHead()	Remove the head node and correctly update head.
A3	deleteTail()	Remove the final node from the list.
A4	modify(int index, T newData)	Replace the data stored at a specified valid index.

With four students, the instructor can assign one different Group A problem to each student.

Part 3 — At-Home Requirement: Complete ALL Group B Problems

After class, complete ALL of the following Group B operations. You have until Tuesday to finish, test, and submit the completed work.

Group B	Suggested Signature	Task
B1	insertAtIndex(int index, Node<T> newNode)	Insert a node at a specified index.
B2	deleteAtIndex(int index)	Delete the node at a specified index.
B3	swap(int index1, int index2)	Swap the DATA stored in two nodes; the nodes themselves do not need to move.
B4	reverse()	Reverse the entire singly linked list by changing the links between nodes.

Part 4 — Generic-Type Testing

Your completed linked-list implementation must be tested with THREE different data types. Use the same MyLinkedList<T> and Node<T> classes for all three.

- MyLinkedList<Integer>
- MyLinkedList<String>
- MyLinkedList<Student>

You do not need a separate linked-list implementation for each data type. The purpose of <T> is to allow the same implementation to work with different object types.

Part 5 — Edge-Case Testing Is Required

A method is not complete just because it works for one normal example. For every method you implement, think about what could be different at the boundaries of the list.

At minimum, test relevant cases from the following list:

- Empty list — What should happen if there is no node to delete, modify, swap, or reverse?
- One-node list — Does the operation leave head in the correct state?
- Head / index 0 — Does an index-based operation work correctly at the first node?
- Tail / last valid index — Does the operation work correctly at the final node?
- Middle of the list — Test a normal internal position.
- Invalid negative index — The program should handle it without corrupting the list.
- Index beyond the list — The program should handle it without corrupting the list.

- For swap: test two different valid indexes and consider what should happen if the indexes are the same.
- For reverse: test an empty list, a one-node list, and a list with several nodes.

For invalid operations: print a clear message such as "Invalid index" or "The list is empty" and leave the list unchanged.

Part 6 — Show BEFORE and AFTER

Your Demo.java should make your tests easy to understand. For each important test, print the list before the operation, identify the operation being tested, and print the list afterward.

Before:

```
10
20
30
```

Operation: insertAtEnd(40)

After:

```
10
20
30
40
```

Index Convention

- The first node is at index 0.
- For modify, delete, and swap, an index must refer to an existing node.
- For insertAtIndex, index 0 inserts at the head.
- For insertAtIndex, inserting immediately after the current last node is allowed as an insertion at the end.

Submission Expectations

Submit your completed project/source files by Tuesday. Your work should include:

- Your assigned Group A method.
- All four Group B methods.
- Tests using Integer, String, and Student lists.
- Normal-case AND edge-case tests.
- Clear output that makes it possible to see whether each operation worked.

What You Should Be Able to Explain

Be prepared to explain your own code, including:

- How you traverse from head to another node.
- Why you sometimes need to stop at the node BEFORE the node being inserted or deleted.
- Which next reference(s) must change for an insertion or deletion.
- Why changing head is a special case.
- How reverse() changes the direction of the links.
- Why the same code works for Integer, String, and Student.

Main Takeaway

Linked-list programming is largely about references. First find the correct node by traversing the list; then carefully change the appropriate data or next reference. Testing boundary cases is essential because head, tail, empty-list, and one-node situations often require special handling.